

Exercices de préparation aux oraux Centrale 2

I. Mercredi 10 juin – 1er groupe

I.1 Énoncé

Soit $f :]-1, 1[\rightarrow \mathbb{R}$.
$$x \mapsto \frac{1}{(1-x^3)(1-x^5)}$$

- 1) a) Montrer que f est développable en série entière au voisinage de 0.

- b) Minorer son rayon de convergence par une constante non nulle.

On note $f(x) = \sum_{n=0}^{+\infty} c_n x^n$.

- 2) Soit, pour tout $n \in \mathbb{N}^*$, $D_n = \{(u, v) \in \mathbb{N}^2, 3u + 5v = n\}$ et $d_n = \text{Card } D_n$.

Y a-t-il un lien entre c_n et d_n ?

- 3) Écrire une fonction **Python** qui calcule les c_n pour tout $n \in \mathbb{N}$.

- 4) Calculer $c_{n+15} - c_n$ avec **Python**, pour tout $n \in \llbracket 0, 200 \rrbracket$.

- 5) Démontrer le résultat obtenu.

- 6) Montrer que le polynôme $(1 - X^3)(1 - X^5)$ divise le polynôme $(1 - X^{15})(1 - X)$.

- 7) Quel est le quotient Q ? Que vaut $Q(1)$? Que vaut $\deg Q$? (on pourra utiliser **Python** pour donner ces résultats).

- 8) Soit $g : x \mapsto (1 - x^{15})f(x) - \frac{1}{1-x}$. Montrer que g est polynomiale sur $] -1, 1[$.

La partie précédente est le sujet d'origine légèrement réécrit.
On pourrait rajouter les questions suivantes :

- 9) Soit $C = \sum_{n=0}^{14} c_n X^n$. Donner les coefficients de C avec **Python**.

- 10) Soit $D = \sum_{n=0}^{14} X^n$. Montrer que pour tout $x \in] -1, 1[$, $f(x) = C + \sum_{n=0}^{+\infty} (C + kD) X^{15k}$.

- 11) En déduire une autre fonction calculant les c_n .

I.2 Corrigé

- 1) a) La fonction $f(x) = \frac{1}{(1-x^3)(1-x^5)}$ est le produit de deux fonctions développables en série entière au voisinage de 0 :

$$\frac{1}{1-x^3} = \sum_{k=0}^{+\infty} x^{3k} \quad \text{et} \quad \frac{1}{1-x^5} = \sum_{m=0}^{+\infty} x^{5m}.$$

Le produit de Cauchy de deux séries entières est une série entière.
Donc f est développable en série entière au voisinage de 0.

- b) Le rayon de convergence de $\frac{1}{1-x^3}$ est 1 (car $|x^3| < 1$ pour $|x| < 1$), et celui de $\frac{1}{1-x^5}$ est aussi 1. Le rayon de convergence de f est donc au moins 1, car le produit de deux séries entières a un rayon de convergence au moins égal au minimum des rayons des deux séries. Ainsi, on peut minorer le rayon de convergence par 1.

- 2) Les coefficients c_n de la série entière de f sont donnés par la formule des coefficients d'un produit de Cauchy.

On obtient que $c_n = \sum_{\substack{k \text{ multiple de } 3 \text{ et } m \text{ multiple de } 5 \\ \text{tel que } k+m=n}} 1 = \sum_{k,m \in \mathbb{N}, 3k+5m=n} 1.$

Autrement dit, c_n est le nombre de solutions entières $(k, m) \in \mathbb{N}^2$ telles que $3k + 5m = n$.

```

3) def calcul_c_n(n) :
    count = 0
    for k in range(n // 3 + 1) :
        for m in range((n - 3 * k) // 5 + 1) :
            if 3 * k + 5 * m == n :
                count += 1
    return count

4) def calcul_difference(n_max=200) :
    differences = []
    for n in range(n_max + 1) :
        c_n = calcul_c_n(n)
        c_n_plus_15 = calcul_c_n(n + 15)
        differences.append(c_n_plus_15 - c_n)
    return differences

```

On observe que $c_{n+15} = c_n + 1$ pour tout $n \in \mathbb{N}$.

- 5) En effet, si $3k + 5m = n$, alors $3(k + 5) + 5m = n + 15$ (type 1) et $3k + 5(m + 3) = n + 15$ (type 2).
Chaque solution pour n donne une solution de type 1 pour $n + 15$.
Toutes les solutions de type 2 sont aussi de type 1, sauf celle pour m maximale, ce qui donne une solution supplémentaire pour $n + 15$.

Si on veut être plus précis : parmi toutes les solutions de D_n , notons (j_m, k_m) celle pour laquelle k_m est le plus grand possible. On observe que si $(j, k) \in D_n$, $(j + 5, k) \in D_{n+15}$. De plus $(j_m, k_m + 3) \in D_{n+15}$ aussi, et ne peut pas être égale à une des solutions précédentes, par maximalité de k_m . Donc $\{(j + 5, k), (j, k) \in D_n\} \sqcup \{(j_m, k_m + 3)\} \subset D_{n+15}$ et ainsi $c_n + 1 \leq c_{n+15}$.

Réciproquement, si (u_m, v_m) est la solution de D_{n+15} pour laquelle v_m est maximale, alors $u_m < 5$ sinon $(u_m - 5, v_m + 3)$ serait aussi une solution et cela contredirait la maximalité de v_m .

Et inversement, si (u, v) est une solution autre que (u_m, v_m) , alors

montrons que $u \geq 5$: si (u_1, v_1) et (u_2, v_2) sont deux solutions distinctes, alors $u_1 \neq u_2$ et $v_1 \neq v_2$. De plus $3|u_1 - u_2| = 5|v_1 - v_2|$, donc 5 divise $3|u_1 - u_2|$. Puisque 3 et 5 sont premiers entre eux, le théorème de Gauss assure que 5 divise $|u_1 - u_2|$. Comme ce dernier terme n'est pas nul, il est supérieur ou égal à 5. Donc si (u, v) est une solution autre que (u_m, v_m) , $|u - u_m| \geq 5$. Or $0 \leq u_m < 5$, donc $u \geq 5$.

Alors $(u - 5, v) \in D_n$,

donc $\{(u - 5, v), (u, v) \in D_{n+15} \setminus \{(u_m, v_m)\}\} \subset D_n$, et ainsi $c_{n+15} - 1 \leq c_n$.

Finalement $c_n = c_{n+15} + 1$.

- 6) 1 est racine double du numérateur et du dénominateur. Comme 3 et 5 sont premiers, les racines du dénominateur (hormis 1) sont simples, et sont toutes racines du numérateur.
7) En utilisant Python, on peut calculer Q explicitement :

```
from numpy.polynomial import Polynomial
```

```

X = Polynomial([0,1])
A = (1-X**15)*(1-X)
B = (1-X**3)*(1-X**5)
Q = A/B

```

On obtient :

$$Q(X) = 1 - X + X^3 - X^4 + X^5 - X^7 + X^8.$$

Ainsi, $Q(1) = 15$ et $\deg Q = 8$.

- 8) On a :

$$(1 - x^{15})f(x) = \frac{1 - x^{15}}{(1 - x^3)(1 - x^5)} = \frac{(1 - x^{15})}{(1 - x^3)(1 - x^5)} = \frac{Q(x)}{1 - x}.$$

Ainsi,

$$g(x) = \frac{Q(x)}{1 - x} - \frac{1}{1 - x} = \frac{Q(x) - 1}{1 - x}.$$

Comme $Q(1) = 0$, le polynôme $Q - 1$ admet 1 comme racine, donc $X - 1$ divise $Q - 1$ et finalement g est bien une fonction polynomiale.

9) $C = 1 + X^3 + X^5 + X^6 + X^8 + X^9 + X^{10} + X^{11} + X^{12} + X^{13} + X^{14}$.

10) Découle de $c_{n+15} = c_n + 1$.

11)

```
C = sum(calcul_c_n(n)*X**n for n in range(15))
D = sum(X**n for n in range(15))
```

```
def calcul_c_n_bis(n) :
    N = n//15
    P = C+(sum((C+k*D)*X**(15*k) for k in range(1,N+1)))
    return P.coef[n]
```

Elle est moins efficace.

II. Mercredi 10 juin – 2nd groupe

II.1 Énoncé

On considère, pour $n \geq 3$, la matrice

$$A_n = \begin{pmatrix} 1 & \cdots & \cdots & \cdots & 1 \\ \vdots & 1 & 0 & \cdots & 0 \\ \vdots & 0 & \ddots & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & 0 \\ 1 & 0 & \cdots & 0 & 1 \end{pmatrix}.$$

- 1) Montrer que A_n est diagonalisable et déterminer son rang.
- 2) On considère la suite $(u_p = \text{tr}(A_3^p))_{p \in \mathbb{N}}$.
 - a) Calculer les quinze premiers termes de cette suite.
 - b) Calculer $B = A_3^3 - 3A_3^2 + A_3 + I_3$.
 - c) Trouver une relation de récurrence simple pour (u_p) . Donner un script en Python permettant de trouver u_p .
- 3)
 - a) Montrer qu'une valeur propre différente de 1 de A_n est solution de l'équation $f_n(x) = 0$, où $f_n(x) = a_n x + b_n + \frac{c_n}{x-1}$ est à déterminer. On s'intéressera au système $A_n X = \lambda X$.
 - b) Montrer que les valeurs propres de A_n sont racines du polynôme $P_n(X) = X^3 - 3X^2 + (4-n)X + (n-2)$.
- 4)
 - a) Montrer que A_n admet trois valeurs propres distinctes $\alpha_n, \beta_n, \gamma_n$.
 - b) Montrer que le système

$$\begin{cases} x + \alpha_n y + \alpha_n^2 z = \alpha_n^4 \\ x + \beta_n y + \beta_n^2 z = \beta_n^4 \\ x + \gamma_n y + \gamma_n^2 z = \gamma_n^4 \end{cases}$$

admet une unique solution (x_n, y_n, z_n) .

- c) Donner un script en python permettant de résoudre ce système (lorsque n est donné).
- d) Quelle vérification matricielle peut-on faire ?

II.2 Corrigé

- 1) Le théorème spectral dit que toute matrice symétrique réelle est diagonalisable, donc A_n l'est.

On effectue l'opération $C_1 \leftarrow C_1 - \sum_{k=2}^n C_k$ et l'on obtient la matrice

$$\begin{pmatrix} -n-2 & 1 & \cdots & \cdots & 1 \\ 0 & 1 & 0 & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ \vdots & & \ddots & \ddots & 0 \\ 0 & \cdots & \cdots & 0 & 1 \end{pmatrix}.$$

Donc $\text{rg } A_n = n$.

- 2) a) On valide `import numpy as np` et on utilise le script suivant :

```
A3 = np.ones((3, 3))
A3[1, 2] = 0
A3[2, 1] = 0
print(A3)
u = [0]*15
M = np.eye(3)
for k in range(15) :
    u[k] = np.trace(M)
    M = np.dot(M, A3)}
print(u)
```

L'exécution python renvoie [3, 3, 7, 15, 35, 83, 199, 479, 1155, 2787, 6727, 16239, 39203, 94643, 228487].

- b) Pour calculer $B = A_3^3 - 3A_3^2 + A_3 + I_3$, on valide

```
A32 = np.dot(A3, A3)    # calcul de A_3^2
A33 = np.dot(A32, A3)   # calcul de A_3^3
B = A33 - 3*A32 + A3 + np.eye(3)
print(B)
```

Python renvoie la matrice nulle.

- c) On multiplie, pour $n \geq 3$, cette relation par A_3^{n-3} et on prend

la trace :

$$\forall n \geq 3, \quad u_n = 3u_{n-1} - u_{n-2} - u_{n-3}.$$

Ceci qui permet un calcul récursif avec python :

```
def u(p) :
    if p == 0 :
        return 3
    if p == 1 :
        return 3
    if p == 2 :
        return 7
    else :
        return 3*u(p - 1) - u(p - 2) - u(p - 3)
```

- 3) a) Soit λ une valeur propre de A_n différente de 1. Il existe une colonne *non nulle* $X = (x_1, x_2, \dots, x_n)^\top$ telle que :

$$\begin{cases} x_1 + x_2 + \cdots + x_n = \lambda x_1 \\ x_1 + x_2 = \lambda x_2 \\ x_1 + x_3 = \lambda x_3 \\ \vdots \\ x_1 + x_n = \lambda x_n. \end{cases}$$

On a donc

$$\begin{cases} (1 - \lambda)x_1 + x_2 + \cdots + x_n = 0 \\ \forall i \in \llbracket 2, n \rrbracket, \quad x_i = \frac{x_1}{\lambda - 1}, \end{cases}$$

ce qui impose que x_1 est non nul (sinon X le serait) et en sommant les $(n - 1)$ dernières équations on obtient

$$1 - \lambda + \frac{n-1}{\lambda-1} = 0.$$

La fonction f_n cherchée est donc $f_n(x) = x - 1 - \frac{n-1}{x-1}$.

- b) Une valeur propre de A_n différente de 1 est donc racine de $(x-1)f_n(x) = x^2 - 2x - n + 2$, polynôme dont 1 n'est pas racine. Or 1 est une valeur propre évidente de A_n ($A_n - I_n$ n'est pas inversible) donc une valeur propre de A_n est solution de $(x^2 - 2x + n)(x-1) = 0$ donc racine du polynôme $P_n(X) = X^3 - 3X^2 + (4-n)X + (n-2)$.

Remarque. Ce résultat est cohérent avec le cas $n = 3$.

- 4) a) Une valeur propre évidente est $\gamma_n = 1$ puisque

$$A - I_n = \begin{pmatrix} 0 & 1 & \cdots & \cdots & 1 \\ 1 & 0 & 0 & \cdots & 0 \\ \vdots & 0 & \ddots & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & 0 \\ 1 & 0 & \cdots & 0 & 0 \end{pmatrix}$$

est de rang 2 donc (sachant que A est diagonalisable) $\gamma_n = 1$ est valeur propre de multiplicité égale à $n-2$. On trouve α_n et β_n , classiquement, avec la trace des deux matrices :

$$A_n = \begin{pmatrix} 1 & \cdots & \cdots & \cdots & 1 \\ \vdots & 1 & 0 & \cdots & 0 \\ \vdots & 0 & \ddots & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & 0 \\ 1 & 0 & \cdots & 0 & 1 \end{pmatrix} \quad \text{et} \quad A_n^2 = \begin{pmatrix} n & 1 & 1 & \cdots & 1 \\ 1 & 2 & 1 & \cdots & 11 \\ 1 & 1 & 2 & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & 1 \\ 1 & 1 & \cdots & 1 & 2 \end{pmatrix}$$

Ce qui donne par similitude

$$\begin{cases} \alpha_n + \beta_n + n - 2 &= n \\ \alpha_n^2 + \beta_n^2 + (n-2)1^2 &= 3n - 2 \end{cases} \quad \text{soit} \quad \begin{cases} \alpha_n + \beta_n &= 2 \\ \alpha_n^2 + \beta_n^2 &= 2n, \end{cases}$$

et en substituant $\beta_n = 2 - \alpha_n$ dans la deuxième équation, on voit que α_n et β_n sont solution de : $x^2 - 2x + 2 - n = 0$. On sait (mais on retrouve aussi) que 1 n'est pas racine de cette équation et le discriminant réduit de cette équation du second degré est $\Delta' = n - 1$. Il est non nul donc $\alpha_n = 1 - \sqrt{n-1}$ et

$\beta_n = 1 + \sqrt{n-1}$ en sont les racines distinctes, et distinctes de 1.

- b) Le système considéré est alors de Cramer (Vandermonde non nul).
- c) Script en python permettant de résoudre ce système, le format n étant donné.

```
import numpy.linalg as alg

n = 8
A = np.eye(n)
for i in range(1, n) :
    A[0, i] = A[i, 0] = 1
print(A)
v = alg.eigvals(A)
print(v) # on remarque que python semble renvoyer dans
        # l'ordre alpha, beta, 1, ..., 1
a,b,c = v[ :3]
M = np.array([[1, a, a**2], [1, b, b**2], [1, c, c**2]])
Y = np.array([a**4, b**4, c**4])
X = alg.solve(M, Y)
print(X)
```

- d) Les valeurs 3 propres de A_n annulent donc le polynôme $X^4 - zX^2 - yX - x$ donc A_n aussi puisqu'elle est diagonalisable. Vérifions-le avec python

```
A2 = np.dot(A, A) # calcul de A_n^2
A3 = np.dot(A2, A) # calcul de A_n^3
A4 = np.dot(A3, A) # calcul de A_n^4
C = A4 - X[2]*A2 - X[1]*A - X[0]*np.eye(n)
print(C)
```

et ça marche !!!